DEPARTMENT OF CS (BS SOFTWARE ENGINEERING)

OOP LAB TERMINAL

COMFLIX MOVIE RECOMMENDATION SYSTEM

PROJECT REPORT

Submitted By:

Ahmed Ali Hashmi (SP25-BSE-004)

Arham Suhail (SP25-BSE-009)

Submitted To:

Mr Saif Ullah Ijaz

Submission Date:

December 17, 2025

REPORT

1. Introduction to COMFLIX

COMFLIX is a flavorful desktop application that simulates a movie recommendation engine. Built entirely in Java, the system uses OOP logic. The application utilizes a layered architecture to separate data management, algorithmic logic, and user interaction. Key features include persistent data storage via binary serialization, dynamic recommendation algorithms (Strategy Pattern), and a responsive Swing-based Graphical User Interface (GUI).

2. System Architecture

The application follows a **Four-Layer Architecture**, making sure of distinct separation of concerns and high maintainability.

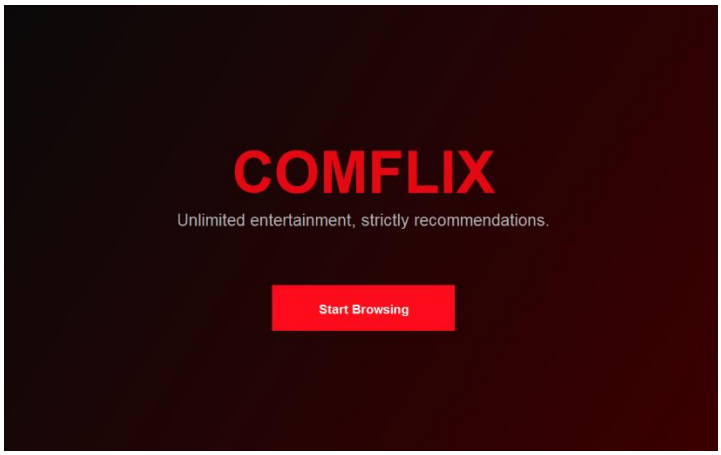
I. The Front-end (UI)

- **Role:** Handles user interactions and data visualization.
- **Purposeful Class:** ComflixGUI
- **Implementation:** Utilizing Java Swing, this layer captures user inputs (via Filters) and renders results dynamically using custom components like MovieCard. It employs lambdas (part of Event Handling) to connect the frontend with the backend logic.

```java
            // Filters Label
            JLabel filtersTitle = new JLabel(text: "FILTERS");
            filtersTitle.setFont(new Font(name: "SansSerif", Font.BOLD, size: 12));
            filtersTitle.setForeground(TEXT_SECONDARY);
            filtersTitle.setAlignmentX(Component.LEFT_ALIGNMENT);
            sidebar.add(filtersTitle);
            sidebar.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));
```

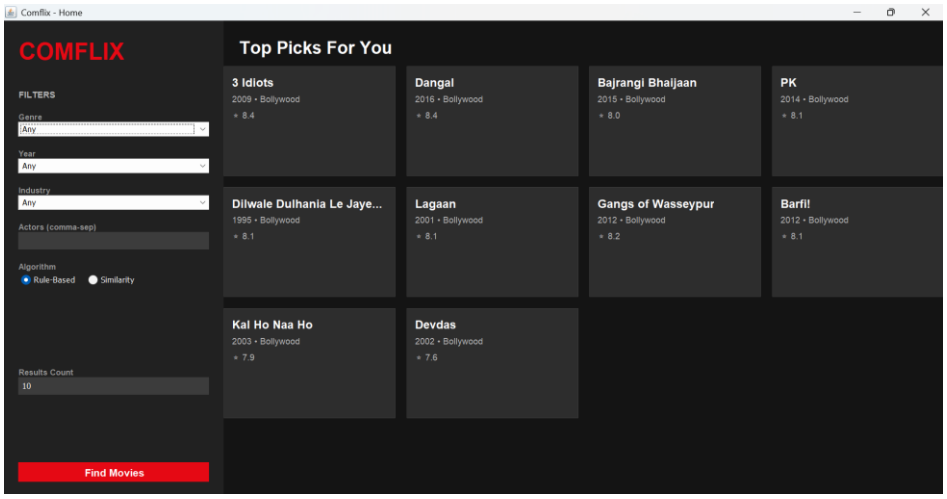
```java
    // Filter Components
    sidebar.add(createFilterLabel(text: "Genre"));
    genreBox = createModernComboBox(new String[]{"Any","Action","Drama","Romance","Sci-Fi","Comedy"
            ,"Thriller","Sports","Animation","Horror"});
    sidebar.add(genreBox);
    sidebar.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));

    sidebar.add(createFilterLabel(text: "Year"));
    yearBox = createModernComboBox(new String[]{"Any","1980-1989","1990-1999","2000-2009","2010-2019"
    sidebar.add(yearBox);
    sidebar.add(Box.createRigidArea(new Dimension(width: 0, height: 15)));
```

a. Welcome Window



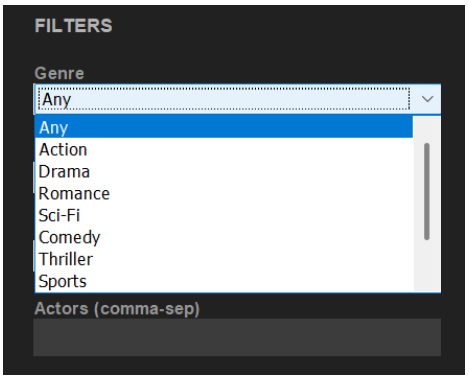
- **Component:** JWindow (or undecorated JFrame).
- **Function:** Acts as the entry point to the application, setting the visual tone.
- **Implementation:** A custom JPanel overrides the paintComponent(Graphics g) method to render a dynamic Red-to-Black linear gradient background. It utilizes a Timer or a "Start" button to transition the user to the main dashboard, ensuring all backend data (deserialization) is loaded before interaction begins.

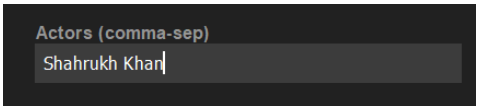
b. Home Page



- **Component:** JFrame (Main Container).
- **Layout:** Uses a BorderLayout. **North:** Contains the "Control Panel" (Logo, Filters, Search Bar). **Center:** A JScrollPane wrapping a JPanel (using GridLayout or FlowLayout) to display the grid of movie results.
- **Role:** Serves as the central hub where the user interacts with the Recommendation Engine.

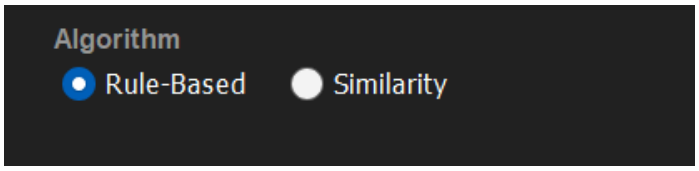
c. Filters





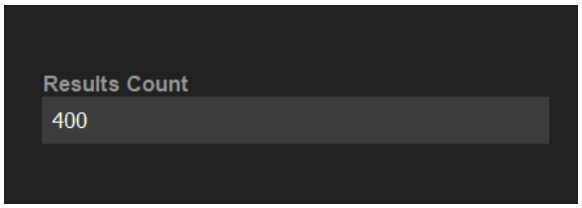
- **Components:** JComboBox (Drop-downs) and JTextField.
- **Function:** Captures user preferences for **Genre**, **Year**, and **Minimum Rating**.
- **Data Flow:** Instead of passing individual strings to the backend, the system aggregates these inputs into a single Filters object. This encapsulates the search criteria, making the method signatures cleaner and easier to maintain.

d. Algorithm Selection



- **Component:** JRadioButton (grouped via ButtonGroup).
- **Function:** Allows the user to select the "Strategy" for recommendations.
  Option A: **Rule-Based** (Simple point system).
  Option B: **Similarity-Based** (Weighted fuzzy matching).
- **Event Handling:** When a radio button is clicked, an ActionListener fires, instantly updating the strategy field within the RecommendationEngine and refreshing the results without needing to restart the app.

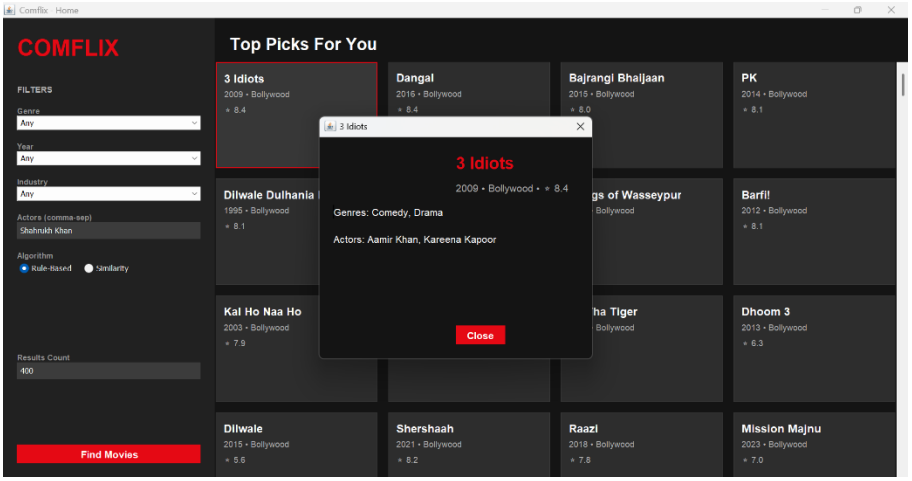
e. Result Count



- **Component:** JLabel (Dynamic Status Bar).
- **Function:** Provides immediate feedback on the search scope (e.g., *"Showing 10 of 450 movies"* vs *"Showing 400 of 450 movies"*).

- **Logic:** This label is updated every time the recommend() method returns a list. It helps the user understand if their filters are too strict (returning 0 results) or too broad.

f. Pop-up Window (Movie Description)



- **Component:** JDialog.
- **Function:** Displays detailed information that doesn't fit on the main movie card (e.g., full plot synopsis, complete cast list, director).
- **Implementation:**

  ***Modality*:** Set as **Modal**, meaning the user must close the details window before returning to the main screen. This focuses the user's attention on the specific movie they selected.

  ***Trigger*:** Activated by clicking a "View Details" button on any MovieCard.

II. The Back-end (OOP Concepts)

- **Role:** The decision-making portion of the application.
- **Purposeful Classes:** RecommendationEngine, RuleBasedRecommendation, SimilarityBasedRecommendation.
- **Implementation:** It processes raw data and allows the system to hot-swap between different recommendation algorithms (e.g., Simple Points vs. Weighted Similarity) without restarting the application.

III. The Domain (Model)

- **Role:** Defines the data structures and business entities.
- **Purposeful Classes:** Media, Movie, User, Filters.
- **Implementation:** These classes use **Encapsulation** to protect data integrity. The Filters class acts as a Data Transfer Object (DTO) to streamline method signatures.

IV. The Data Storage

- **Role:** Manages long-term data storage.
- **Purposeful Classes:** MovieSeeder, movies.dat.
- **Implementation:** Uses Java I/O Streams (ObjectOutputStream) to serialize the object hierarchy into a binary file, producing a local database.

3. Core OOP Implementation Details

A. Inheritance & Abstraction

The data model relies on a hierarchical structure for code reuse.

- **Class Media (Abstract):** Defines properties (id, title, year) common to all media types. It is declared abstract to prevent sudden instantiation.

```java
public abstract class Media implements Serializable {
    private static final long serialVersionUID = 1L;

    protected int id;
    protected String title;
    protected int releaseYear;
```

- **Class Movie (Concrete):** Extends Media, adding specific fields like genres and actors.

```java
public class Movie extends Media implements Serializable {
    private static final long serialVersionUID = 2L;

    private ArrayList<String> genres;   // e.g., Action, Drama
    private ArrayList<String> actors;   // e.g., "Shah Rukh Khan"
    private double rating;              // 0.0 - 10.0
    private String industry;            // Hollywood, Bollywood, etc.
```

- **Benefit:** If the system expands to include TVShow or Documentary, they simply extend Media without rewriting base logic.

B. Polymorphism

Polymorphism is used to treat different objects and algorithms uniformly.

- **Data Polymorphism:** The Movie class overrides getDetails() and toString() to provide custom string representations for the UI.

```java
    @Override
    public String getDetails() {
        return String.format(format: "%s (%d) — %.1f — %s — %s",
                title, releaseYear, rating, industry, String.join(delimiter: ", ", genres));
    }

    @Override
    public String toString() {
        return String.format(format: "Movie{id=%d, title='%s', year=%d, rating=%.1f, industry=%s}"
                id, title, releaseYear, rating, industry);
    }
```

- **Algorithmic Polymorphism:** The RecommendationAlgorithm interface allows the RecommendationEngine to accept any algorithm implementation (RuleBased or SimilarityBased) interchangeably.

C. Encapsulation

All data fields within User, Movie, and Filters are private. Access is strictly controlled via public getters and setters.

```java
    // encapsulation
    public int getId() { return id; }
    public String getTitle() { return title; }
    public int getReleaseYear() { return releaseYear; }
    public void setId(int id) { this.id = id; }
    public void setTitle(String title) { this.title = title; }
    public void setReleaseYear(int year) { this.releaseYear = year; }
```

- **Deep Copying:** The User.addWatchedMovie(Movie m) method utilizes a custom deepCopy() method. This ensures that modifying a movie in a user's list does not corrupt the original movie object in the main database.

4. Algorithmic Logic

The main intelligence (or usefulness per se) of COMFLIX lies in its recommendation engine, which supports two distinct and extremely interesting strategies:

Algorithm Type	Logic Description
Rule-Based	Assigns fixed integers (e.g., +5 points for a genre match). It's mostly for simple, predictable results.
Similarity-Based	Calculates a general match ratio (0.0 to 1.0) based on weighted criteria (Genre: 45%, Recency: 5%). It's useful for nuanced recommendations.

The Actual Code Snippet:

```java

    public void setAlgorithm(RecommendationAlgorithm algorithm) { this.algorithm = algorithm; }
```

5. Serialization

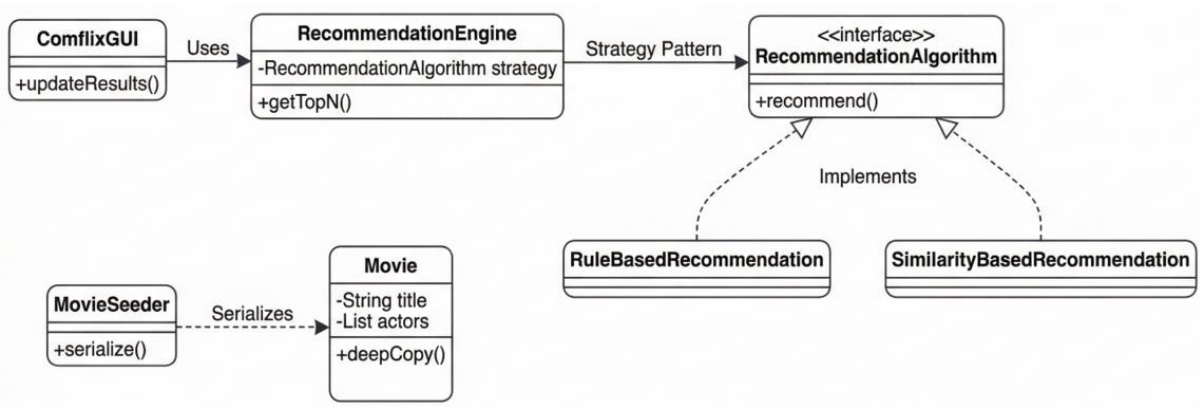
To persist data between sessions without a heavy SQL database, we used Java Serialization (Input/Output Stream).

- **Writing:** The MovieSeeder class instantiates objects and writes them to movies.dat using ObjectOutputStream.

- **Reading:** The system uses ObjectInputStream to deserialize the byte stream back into an ArrayList<Movie>.
- **Safety:** Exception handling catches FileNotFoundException and ClassNotFoundException to prevent crashes during the loading phase.

6. Visual System Design

Below is the UML representation of the system's class structure.



7. Conclusion

The Comflix project successfully implements a full-stack desktop solution. It moves beyond simple scripting to demonstrate architectural proficiency. By combining **Persistence**, **Polymorphism**, and **GUI Event Handling**, the system meets the complex requirements of a modern software application while maintaining clean, modular, and readable code.